Low-Level Design (LLD) Specification

Modules, Interfaces, Class Models, Sequence Flows, and Design Patterns

1. Module Organization & Design Patterns

DocRAG enforces Clean Architecture with strict separation of concerns:

- **Repository / Service Pattern:** Business logic in services/ is isolated from FastAPI routers in api/v1/ and SQLAlchemy ORM models in models/.
- **Strategy Pattern:** StorageProvider interface defines load/save/delete/exists, implemented by LocalStorageProvider and S3StorageProvider.
- **Factory Pattern:** EmbeddingService and RAGService dynamically instantiate provider clients (Groq vs Gemini, FastEmbed vs Cloud) based on configuration.

2. Core Service Contracts

ExtractionService Contract

```
class ExtractionService:
    @classmethod
    def extract(cls, file_bytes: bytes, filename: str) -> List[ExtractedPage]: ...
    @staticmethod
    def validate_file_content(file_bytes: bytes, filename: str) -> str: ...
```

ChunkingService Contract

```
class ChunkingService:
    def __init__(self, chunk_size: int = 800, chunk_overlap: int = 150): ...
    def chunk_pages(self, pages: List[ExtractedPage]) -> List[ChunkData]: ...
```

RAGService Contract

```
class RAGService:
    async def retrieve_chunks(self, db, user_id, query, document_ids, top_k, threshold) -> List[
Tuple]: ...
    async def answer_query(self, db, user_id, question, document_ids, history, top_k, threshold)
 -> Tuple[str, List[Citation]]: ...
```

3. State Transition Model

The document ingestion lifecycle follows deterministic state transitions:

- **uploaded:** Original file validated, stored in storage, document metadata committed in PostgreSQL.
- **queued:** Ingestion task dispatched to Redis queue.
- **processing:** Celery worker acquired job, actively extracting text and generating vectors.
- **completed:** All chunks and vector embeddings persisted atomically; page_count and chunk_count updated.

- **failed:** Permanent error encountered (corrupt file, empty content). Descriptive error_message saved.